

Laboratorio IA 3: Navegacion evolutiva

Materia: Inteligencia Artificial

Dinamica: practica en terminal **Entregable:** experimento, codigo y conclusiones

Extender el robot evolutivo para navegar por una cuadrícula con obstáculos. En esta sesión se estudia cómo la representación, la función de fitness y el cruce genético cambian el resultado de la evolución.

1. De la clase 2 a la clase 3

En la clase anterior la población podía moverse libremente por la cuadrícula y se comparaban dos presiones: explorar soluciones nuevas o explotar las mejores conocidas. Ahora el entorno tiene obstáculos. Una secuencia de movimientos ya no vale solo por acercarse a la meta: también debe evitar choques y aprovechar sus pasos.

El algoritmo no define por sí solo qué significa una buena solución. Esa decisión aparece en la función de fitness. Si el fitness premia una conducta equivocada, la población puede evolucionar rápidamente hacia una respuesta que no resuelve el problema.

2. Preparar y ejecutar

Coloca `robot_obstaculos.py` en la misma carpeta que este manual y ejecuta:

```
python3 robot_obstaculos.py
```

Para repetir exactamente el experimento y omitir las pausas de la animación:

```
python3 robot_obstaculos.py --semilla 7 --sin-pausa
```

También puedes ejecutar cada variante por separado:

```
python3 robot_obstaculos.py --modo mutacion --semilla 7 --sin-pausa
python3 robot_obstaculos.py --modo cruce --semilla 7 --sin-pausa
```

3. Como representa el problema

Cada robot es una lista de 18 comandos: U, D, L y R. El robot comienza en (0, 0) y debe llegar a (7, 7). Los obstáculos no se pueden atravesar: si un comando intenta entrar en un obstáculo, el robot conserva su posición y registra un choque.

La animación conserva la convención visual de las clases anteriores. La leyenda de la terminal es: robot, meta alcanzada, meta, obstáculo, inicio y celda libre. Los símbolos aparecen exactamente en el programa incluido al final: `robot_obstaculos.py`.

El programa compara dos estrategias:

- **Solo mutacion:** se elige un padre y se cambian algunos comandos.
- **Mutacion + cruce:** se combinan dos padres en un punto y luego se mutan algunos comandos del hijo.

El cruce de un punto conserva el comienzo de un padre y combina el final del otro:

```
hijo = primer_padre[:punto] + segundo_padre[punto:]
```

4. La funcion de fitness

El puntaje comienza en 500 y se modifica con estas reglas:

- distancia a la meta: -35 por unidad;
- paso util: $+4$;
- choque contra obstaculo: -30 ;
- visita repetida: -3 ;
- llegada a la meta: $+2000$.

Estas cantidades no son verdades universales. Son una hipotesis sobre que conducta conviene favorecer. Cambiarlas modifica la presion selectiva.

5. Experimento guiado

1. Antes de ejecutar, predice que variante llegara antes y cual conservara mas diversidad.
2. Ejecuta el modo comparativo con las semillas 7, 21 y 42.
3. Completa una tabla con llegada, generaciones, puntaje, choques, pasos utiles y diversidad final.
4. Compara los resultados de varias semillas. No uses una sola corrida para afirmar que un metodo es mejor.
5. Cambia una sola penalizacion del fitness y repite el experimento.

Semilla	Metodo	Llego	Generaciones	Puntaje	Choques	Diversidad
7	Solo mutacion					
7	Mutacion + cruce					
21	Solo mutacion					
21	Mutacion + cruce					
42	Solo mutacion					
42	Mutacion + cruce					

1. Por que un robot cercano a la meta puede tener peor puntaje que otro mas lejano?
2. Que ventaja aporta combinar dos padres en lugar de copiar uno solo?
3. Que ocurre si la penalizacion por choques es demasiado pequena? Y si es demasiado grande?
4. La mayor diversidad final implica necesariamente una mejor solucion?
5. Que parte del programa representa el problema y que parte representa el algoritmo de busqueda?

6. Reto de modificacion

Elige una sola modificacion y conserva las demas condiciones del experimento:

- cambia la posicion de dos obstaculos sin bloquear completamente el mapa;
- modifica la longitud del ADN y explica el efecto sobre las rutas;
- implementa cruce uniforme como alternativa al cruce de un punto;
- cambia una recompensa o penalizacion del fitness y justifica la nueva presion selectiva;
- registra la mejor puntuacion de cada generacion y describe su evolucion.

7. Entrega

Entrega el archivo Python modificado, la tabla con tres semillas y una conclusion de 8 a 10 lineas. La conclusion debe responder: *como influyeron los obstaculos, el fitness y el cruce en el comportamiento de la poblacion?*

8.Codigo completo

El codigo se inserta directamente desde el archivo de la practica para mantener sincronizados el manual y el programa.

```
import argparse
import os
import random
import time

# =====
# 1. CONFIGURACION DEL PROBLEMA
# =====
# Cada robot tiene un ADN formado por comandos de movimiento.
COMANDOS = ("U", "D", "L", "R")

# La cuadrícula usa coordenadas (fila, columna), empezando en (0, 0).
TAMANO = 8
INICIO = (0, 0)
META = (7, 7)

# Estas celdas estan bloqueadas. El robot no puede entrar en ellas.
OBSTACULOS = {
    (0, 3), (1, 3), (2, 1), (2, 2), (2, 3),
    (3, 5), (4, 5), (5, 2), (5, 3), (5, 4),
    (6, 6),
}

# Parametros de la poblacion y de los individuos.
LONGITUD_ADN = 18
TAMANO_POBLACION = 100
ELITE = 10
PADRES = 40

# Los escapes Unicode mantienen el archivo en ASCII, pero la terminal
# sigue mostrando los mismos emojis que en las clases anteriores.
EMOJI_ROBOT = "\U0001f916"
EMOJI_META = "\U0001f7e9"
EMOJI_META_ALCANZADA = "\U0001f389"
EMOJI_OBSTACULO = "\u2b1b"
EMOJI_INICIO = "\U0001f535"
EMOJI_LIBRE = "\u2b1c"
```

```

# =====
# 2. SIMULACION DEL ENTORNO
# =====
def limpiar_pantalla():
    """Limpia la terminal antes de dibujar el siguiente paso."""
    os.system("cls" if os.name == "nt" else "clear")

def mover(posicion, comando):
    """Intenta ejecutar un comando y devuelve posicion y resultado."""
    fila, columna = posicion
    cambios = {"U": (-1, 0), "D": (1, 0), "L": (0, -1), "R": (0, 1)}
    cambio_fila, cambio_columna = cambios[comando]
    destino = (fila + cambio_fila, columna + cambio_columna)

    # Un movimiento fuera del tablero no cambia la posicion.
    if not (0 <= destino[0] < TAMANO and 0 <= destino[1] < TAMANO):
        return posicion, "borde"

    # Chocar con un obstaculo tampoco permite avanzar.
    if destino in OBSTACULOS:
        return posicion, "obstaculo"

    return destino, "avance"

def recorrer(adn):
    """Ejecuta un ADN y registra trayectoria, choques y pasos utiles."""
    posicion = INICIO
    trayectoria = [posicion]
    choques = 0
    pasos_utiles = 0

    for comando in adn:
        nueva_posicion, resultado = mover(posicion, comando)
        if resultado == "obstaculo":
            choques += 1
        if resultado == "avance":
            pasos_utiles += 1
        posicion = nueva_posicion
        trayectoria.append(posicion)

    return trayectoria, choques, pasos_utiles

# =====
# 3. REGLA DE SUPERVIVENCIA: EL FITNESS
# =====
def evaluar(adn):
    """Calcula el fitness de un robot: cuanto mayor, mejor solucion."""
    trayectoria, choques, pasos_utiles = recorrer(adn)
    posicion = trayectoria[-1]
    distancia = abs(META[0] - posicion[0]) + abs(META[1] - posicion[1])
    visitas_repetidas = len(trayectoria) - len(set(trayectoria))

    # Los pesos son decisiones del disenador y pueden modificarse.
    puntaje = 500
    puntaje -= distancia * 35
    puntaje += pasos_utiles * 4
    puntaje -= choques * 30
    puntaje -= visitas_repetidas * 3

    # Llegar vale mucho mas que estar simplemente cerca.

```

```

    if posicion == META:
        puntaje += 2000
    return puntaje

def diversidad(poblacion):
    """Cuenta cuantos ADN diferentes hay en la poblacion."""
    return len({"".join(adn) for adn in poblacion})

# =====
# 4. OPERADORES GENETICOS
# =====
def mutar(adn, tasa_mutacion, rng):
    """Cambia algunos genes al azar segun la tasa de mutacion."""
    return [
        rng.choice(COMANDOS) if rng.random() < tasa_mutacion else comando
        for comando in adn
    ]

def cruzar(primer_padre, segundo_padre, rng):
    """Combina el comienzo de un padre y el final del otro."""
    punto = rng.randint(1, len(primer_padre) - 1)
    return primer_padre[:punto] + segundo_padre[punto:]

# =====
# 5. CICLO EVOLUTIVO
# =====
def evolucionar(tasa_mutacion, usar_cruce, semilla, maximo_generaciones=3000):
    """Ejecuta el algoritmo genetico y devuelve el mejor resultado."""
    rng = random.Random(semilla)

    # La primera generacion es completamente aleatoria.
    poblacion = [
        [rng.choice(COMANDOS) for _ in range(LONGITUD_ADN)]
        for _ in range(TAMANO_POBLACION)
    ]
    historial = []

    for generacion in range(maximo_generaciones + 1):
        # El primer robot sera el mejor despues de ordenar la poblacion.
        poblacion.sort(key=evaluar, reverse=True)
        mejor = poblacion[0]
        puntaje = evaluar(mejor)
        trayectoria, choques, pasos utiles = recorrer(mejor)
        historial.append((generacion, puntaje, diversidad(poblacion)))

        # Terminamos temprano si ya encontramos una ruta valida.
        if trayectoria[-1] == META:
            return resultado(
                usar_cruce, mejor, generacion, puntaje, diversidad(poblacion),
                choques, pasos utiles, historial,
            )

        # El elitismo copia directamente a los mejores robots.
        nueva_poblacion = [robot[:] for robot in poblacion[:ELITE]]
        while len(nueva_poblacion) < TAMANO_POBLACION:
            # Elegimos padres de la parte superior de la poblacion.
            primer_padre = rng.choice(poblacion[:PADRES])
            if usar_cruce:
                segundo_padre = rng.choice(poblacion[:PADRES])
                hijo = cruzar(primer_padre, segundo_padre, rng)
            else:

```

```

        hijo = primer_padre[:]

        # El cruce combina; la mutacion introduce variacion nueva.
        nueva_poblacion.append(mutar(hijo, tasa_mutacion, rng))
    poblacion = nueva_poblacion

    # Si se agotan las generaciones, devolvemos el mejor disponible.
    poblacion.sort(key=evaluar, reverse=True)
    mejor = poblacion[0]
    trayectoria, choques, pasos utiles = recorrer(mejor)
    return resultado(
        usar_cruce, mejor, maximo_generaciones, evaluar(mejor),
        diversidad(poblacion), choques, pasos utiles, historial,
    )

def resultado(usar_cruce, adn, generacion, puntaje, diversidad_final,
             choques, pasos utiles, historial):
    """Reune las metricas que compararemos en el experimento."""
    trayectoria, _, _ = recorrer(adn)
    return {
        "metodo": "mutacion + cruce" if usar_cruce else "solo mutacion",
        "adn": adn,
        "generacion": generacion,
        "puntaje": puntaje,
        "diversidad": diversidad_final,
        "choques": choques,
        "pasos utiles": pasos utiles,
        "llego": trayectoria[-1] == META,
        "historial": historial,
    }

# =====
# 6. VISUALIZACION Y CLI
# =====
def dibujar(resultado, pausa):
    """Anima la trayectoria del mejor robot sobre la cuadrícula."""
    adn = resultado["adn"]
    trayectoria, _, _ = recorrer(adn)

    for paso, posicion in enumerate(trayectoria[1:], start=1):
        limpiar_pantalla()
        print(
            f"{{resultado['metodo']}.upper()}} | "
            f"Gen {{resultado['generacion']}} | Puntos: {{resultado['puntaje']}}"
        )
        print(
            f"{{EMOJI_ROBOT}} = robot | {{EMOJI_META_ALCANZADA}} = meta alcanzada | "
            f"{{EMOJI_META}} = meta | {{EMOJI_OBSTACULO}} = obstaculo"
        )
        print(f"{{EMOJI_INICIO}} = inicio | {{EMOJI_LIBRE}} = celda libre\n")
        for fila in range(TAMANO):
            linea = ""
            for columna in range(TAMANO):
                celda = (fila, columna)
                if celda == posicion:
                    linea += EMOJI_ROBOT
                elif celda == INICIO:
                    linea += EMOJI_INICIO
                elif celda == META:
                    linea += EMOJI_META_ALCANZADA if posicion == META else EMOJI_META
                elif celda in OBSTACULOS:
                    linea += EMOJI_OBSTACULO
            else:

```

```

        linea += EMOJI_LIBRE
    print(linea)
    print(
        f"Paso {paso}/{len(adn)} | ADN: {''.join(adn)} | "
        f"Choques: {resultado['choques']}"
    )
    if pausa:
        time.sleep(pausa)

def mostrar_resultado(resultado, pausa):
    """Muestra la animacion y las metricas finales de una variante."""
    dibujar(resultado, pausa)
    estado = "LLEGO" if resultado["llego"] else "NO LLEGO"
    print(f"\nResultado: {estado}")
    print(f"Pasos utiles: {resultado['pasos_utiles']}")
    print(f"Diversidad final: {resultado['diversidad']}/{TAMANO_POBLACION}")
    if pausa:
        time.sleep(1)

def main():
    """Lee las opciones de terminal y ejecuta el experimento solicitado."""
    parser = argparse.ArgumentParser(
        description="Compara mutacion y cruce en una poblacion de robots."
    )
    parser.add_argument(
        "--modo",
        choices=("comparar", "mutacion", "cruce"),
        default="comparar",
    )
    parser.add_argument("--semilla", type=int, default=7)
    parser.add_argument("--sin-pausa", action="store_true")
    args = parser.parse_args()

    # comparar ejecuta ambas variantes; los otros modos ejecutan solo una.
    experimentos = {
        "mutacion": ("solo mutacion", False),
        "cruce": ("mutacion + cruce", True),
    }
    modos = tuple(experimentos) if args.modo == "comparar" else (args.modo,)
    resultados = []

    for indice, modo in enumerate(modos):
        _, usar_cruce = experimentos[modo]

        # Al comparar, usamos semillas distintas para las dos variantes.
        resultado_actual = evolucionar(
            tasa_mutacion=0.08,
            usar_cruce=usar_cruce,
            semilla=args.semilla + indice,
        )
        resultados.append(resultado_actual)
        mostrar_resultado(resultado_actual, 0 if args.sin_pausa else 0.08)

    if len(resultados) == 2:
        print("\nCOMPARACION")
        print("Metodo          Llego  Generaciones  Puntaje  Choques  Diversidad")
        print("-----")
        for resultado_actual in resultados:
            llego = "si" if resultado_actual["llego"] else "no"
            print(
                f"{resultado_actual['metodo']:<20}{llego:<7}"
                f"{resultado_actual['generacion']:<15}"
                f"{resultado_actual['puntaje']:<9}"
            )

```

```
        f"{resultado_actual['choques']:<9}"  
        f"{resultado_actual['diversidad']}/{TAMANO_POBLACION}"  
    )  
  
if __name__ == "__main__":  
    main()
```